Experiment 17: RNN Gradient Analysis and Gradient Clipping

Question

Problem Statement: A text-generation model at a content start-up produces NaN losses midway through training because of exploding gradients. A character-level text corpus (for example, a public-domain novel) trained with a deliberately high learning rate. The objective is to detect exploding gradients and stabilise training with gradient clipping. Use TensorFlow, Keras, NumPy, Matplotlib.

Tasks:

1. Train an RNN without clipping and log the global gradient norm each step.
2. Identify the steps where the gradient spikes or loss becomes NaN.
3. Retrain with clipnorm and with clipvalue at different thresholds.
4. Plot gradient norm and loss curves for each setting.
5. Recommend a clipping threshold.

Code

```
"""Experiment 17: RNN Gradient Analysis and Gradient Clipping (character-level text model)"""
import os
import urllib.request
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# ---------------- Data: public-domain novel (Alice in Wonderland, Project Gutenberg) ----------------
URL = "https://www.gutenberg.org/files/11/11-0.txt"
FALLBACK = ("alice was beginning to get very tired of sitting by her sister on the bank, and of having "
            "nothing to do: once or twice she had peeped into the book her sister was reading, but it had "
            "no pictures or conversations in it, and what is the use of a book, thought alice, without "
            "pictures or conversations? ") * 60

def load_text():
    if os.path.exists("novel.txt"):                                  # your own text file
        return open("novel.txt", encoding="utf-8", errors="ignore").read()
    try:
        return urllib.request.urlopen(URL, timeout=15).read().decode("utf-8", errors="ignore")
    except Exception as e:
        print("WARNING: could not download the novel (", e, ") -> using a short built-in text instead.")
        return FALLBACK

text = load_text().lower()[:100000]
chars = sorted(set(text)); c2i = {c: i for i, c in enumerate(chars)}
data = np.array([c2i[c] for c in text]); V = len(chars)
print("Corpus length:", len(text), "| vocabulary size:", V)

SEQ, BATCH, STEPS, LR = 40, 32, 300, 0.5      # deliberately high learning rate (plain SGD)

def get_batch(rng):
    idx = rng.randint(0, len(data) - SEQ - 1, BATCH)
    x = np.stack([data[i:i + SEQ] for i in idx]); y = data[idx + SEQ]   # predict the next character
    return tf.constant(x), tf.constant(y)

def build():
    return keras.Sequential([layers.Input((SEQ,)), layers.Embedding(V, 32),
                             layers.SimpleRNN(128, activation="relu"), layers.Dense(V)])

loss_fn = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

def train(mode=None, thr=None):
    tf.random.set_seed(0); rng = np.random.RandomState(0)              # identical start & batches for every
run
    model = build(); opt = keras.optimizers.SGD(LR)
    norms, losses = [], []
    for step in range(STEPS):
```

```
        x, y = get_batch(rng)
        with tf.GradientTape() as tape:
            loss = loss_fn(y, model(x, training=True))
        grads = tape.gradient(loss, model.trainable_variables)
        gnorm = float(tf.linalg.global_norm(grads))                    # Task 1: log global gradient norm
        if mode == "norm":                                             # clipnorm: rescale if norm > thr
            grads, _ = tf.clip_by_global_norm(grads, thr)
        elif mode == "value":                                          # clipvalue: clip each element to [-
thr, thr]
            grads = [tf.clip_by_value(g, -thr, thr) for g in grads]
        opt.apply_gradients(zip(grads, model.trainable_variables))
        norms.append(gnorm); losses.append(float(loss))
    return np.array(norms), np.array(losses)

# ---------------- Tasks 1 & 2: no clipping ----------------
norms0, loss0 = train()
nan_steps = np.where(~np.isfinite(loss0))[0]
spikes = np.where(np.isfinite(norms0) & (norms0 > 10 * np.nanmedian(norms0[:20])))[0]
print(f"\nNo clipping: first NaN loss at step {nan_steps[0] if len(nan_steps) else 'never'}")
print(f"Gradient spike steps (norm > 10x early median): {spikes[:10]}{' ...' if len(spikes) > 10 else ''}")
print(f"Max finite gradient norm: {np.nanmax(norms0[np.isfinite(norms0)]):.2f}")

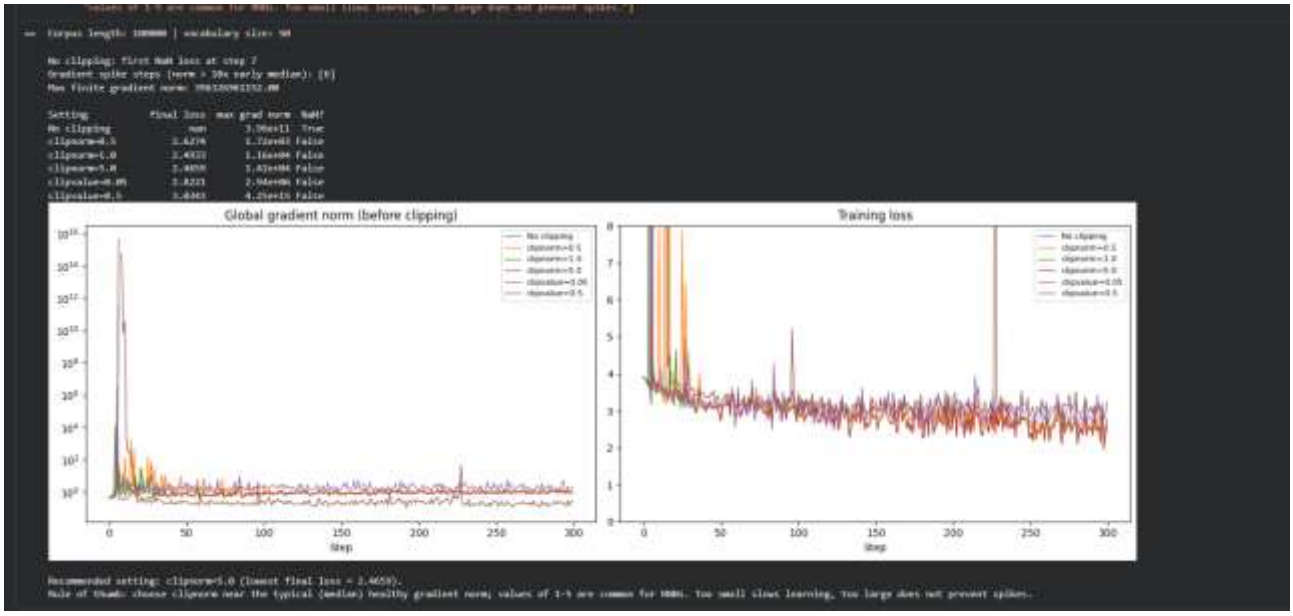
# ---------------- Task 3: clipping with different thresholds ----------------
settings = [("No clipping", None, None), ("clipnorm=0.5", "norm", 0.5), ("clipnorm=1.0", "norm", 1.0),
            ("clipnorm=5.0", "norm", 5.0), ("clipvalue=0.05", "value", 0.05), ("clipvalue=0.5", "value",
0.5)]
runs = {"No clipping": (norms0, loss0)}
for name, mode, thr in settings[1:]:
    runs[name] = train(mode, thr)

print(f"\n{'Setting':16}{'final loss':>12}{'max grad norm':>15}{'NaN?':>6}")
final = {}
for name, (n, l) in runs.items():
    fin = np.mean(l[-20:]) if np.all(np.isfinite(l)) else np.nan
    final[name] = fin
    print(f"{name:16}{fin:12.4f}{np.nanmax(n[np.isfinite(n)]) if np.isfinite(n).any() else
np.nan:15.3g}{str(not np.all(np.isfinite(l))):>6}")

# ---------------- Task 4: plots ----------------
fig, ax = plt.subplots(1, 2, figsize=(15, 5))
for name, (n, l) in runs.items():
    ax[0].plot(n, label=name, lw=1); ax[1].plot(l, label=name, lw=1)
ax[0].set_yscale("log"); ax[0].set_title("Global gradient norm (before clipping)"); ax[0].set_xlabel("Step")
ax[1].set_title("Training loss"); ax[1].set_xlabel("Step"); ax[1].set_ylim(0, 8)
ax[0].legend(fontsize=8); ax[1].legend(fontsize=8)
plt.tight_layout(); plt.show()

# ---------------- Task 5: recommendation ----------------
valid = {k: v for k, v in final.items() if np.isfinite(v) and k != "No clipping"}
best = min(valid, key=valid.get)
print(f"\nRecommended setting: {best} (lowest final loss = {valid[best]:.4f}).")
print("Rule of thumb: choose clipnorm near the typical (median) healthy gradient norm; "
      "values of 1-5 are common for RNNs. Too small slows learning, too large does not prevent spikes.")
```

Output


Global gradient norm (before clipping)
Training loss
Step
Step